

Úvod do databázových systémů

Kapitola 6
Relační algebra

Manipulace s relacemi

- zatím máme struktury (relace), ale nemáme operace nad nimi
- jsou nutné prostředky pro
 - manipulace s relacemi
 - vložení n-tice do dané relace
 - zrušení/změna daných n-tic v dané relaci
 - vždy nutno kontrolovat deklarativní integritní omezení!
 - dotazování
 - pomocí nějakého jazyka (relační algebra a kalkul, SQL)
- **množinový přístup**
 - vstupem operace jsou **celé** relace
 - výstupem operace je opět relace

Manipulace s relacemi

- databázový **dotaz** nad schématem S je výraz, který vrátí odpověď se schématem T, přitom
 - *definičním oborem* jsou všechna úložiště se schématem S
 - *oborem hodnot* jsou všechny relace se schématem T
 - data v odpovědi pocházejí z databáze
 - odpověď nezávisí na fyzickém uložení dat
 - netřeba se starat o umístění relací či fyz. strukturu dat
- **dotazovací jazyk** je množina všech použitelných výrazů pro konstrukci dotazu

Manipulace s relacemi

- aktualizace relace
 - přidání n-tice do množiny (INSERT)
 - odebrání n-tice z množiny (DELETE)
 - změna n-tice v množině (UPDATE)
- operace DELETE a UPDATE vyžadují identifikaci n-tic, na něž se vztahují
 - identifikace na základě hodnot atributů (žádné jiné informace o prvcích v množině totiž nemáme)
- operace INSERT a UPDATE sledují unikátnost n-tic (množiny...)

Manipulace s relacemi

- základní ideje RMD z hlediska operace s daty
 - 1. normální forma – komponenty n-tic jsou atomické
 - jedinečnost n-tic – množinový přístup
 - způsob uložení dat není podstatný
 - relace jsou identifikovány jménem, to je vše, co potřebujeme
 - pořadí atributů v relaci není podstatné
 - atributy jsou identifikovány jménem
 - pořadí n-tic v relacích není definované
 - n-tice jsou identifikovány hodnotami
 - viz další snímek...

Manipulace s relacemi

- základní ideje RMD z hlediska operace s daty
 - přístup k n-ticím relace se děje **výhradně** na základě jejich obsahu
 - tj. nelze třeba říct, že odstraníme třetí řádek tabulky
 - SŘBD sice mají identifikátory řádků tabulek, ale ty by se neměly v běžné praxi používat, pouze při správě DB
 - n-tice vybíráme na základě logické podmínky, testující hodnoty dat v n-ticích
 - například řekneme: chci pracovat s n-ticí, která má v atributu „rodné číslo“ hodnotu 806212/0875

Relační algebra

- množina operací, jejichž aplikace na nějaké relace vrací opět relaci
 - výsledek by měl mít definováno schéma
 - výsledkem dotazu je relace, která může být vstupem do dalšího dotazu – dotazy se dají vnořovat a řetězit
 - nspecifikujeme, jak se mají věci udělat, ale co má být výsledkem
 - řeší pouze dotazování, nikoliv manipulaci s daty
- relační algebra byla vzorem pro návrh příkazu `SELECT` v SQL
 - `SELECT` umí v současnosti více než RA (agregace, vnější spojení, vnořené dotazy, řazení).

Relační algebra

- daná operace je popsána výrazem
 - např. $(R \cap S) \times W$
- je třeba rozlišit pojmy **dotaz** a **výraz**
 - dotaz je funkce na stavech databáze
 - výraz je pouze syntaktický zápis dotazu
 - týž dotaz může být popsán více výrazy
- jestliže dva různé výrazy označují tentýž dotaz, jsou **ekvivalentní**
 - např. v algebře je ekvivalentní $(x * x)$ a (x^2)
 - ekvivalentní výrazy jsou také $(R \cap S) \times W$ a $(R - (R - S)) \times W$

Relační algebra

- výrazy (dotazy) se skládají z operací a operandů
 - operandem je vždy relace
 - relace jsou množiny, máme proto k dispozici běžné množinové operace
 - součin ($\times$)
 - sjednocení ($\cup$)
 - průnik ($\cap$)
 - rozdíl ($-$)
 - na součin se neklade žádné omezení, ostatní operace jen s kompatibilními operandy
 - stejný počet atributů, rovnost odpovídajících domén

Relační algebra

■ další operace

- přejmenování atributů (například při kartézském součinu relace sama se sebou)
- **projekce**: $R[C]$ relace se schematem $R(A)$ na množinu atributů C , kde $C \subseteq A$
- **selekce** (restrikce): $R(\varphi)$ relace se schematem $R(A)$ podle logické podmínky φ
- **přirozené spojení** relací R, S se schématy $R(A), S(B)$: $R * S$

■ minimální množina operací

- přejmenování, selekce, projekce, $\times$, $\cup$, $-$
- ostatní (třeba $*$) se dají odvodit

Relační algebra

- projekce $R[C]$ relace se schematem $R(\mathbf{A})$ na množinu atributů C , kde $C \subseteq \mathbf{A}$
 - projekce n -tice $u[C]$ – z n -tice vybereme pouze atributy, patřící do množiny C
 - je-li R relace, pak $R[C] = \{ u[C] \mid u \in R \}$
 - vytvoří relaci se schematem C a n -ticemi, které vzniknou z původní relace odstraněním hodnot atributů z $\mathbf{A} - C$. Odstraněny jsou i event. duplicitní n -tice
 - přijmeme-li na chvíli analogii relace a tabulky, jde o „svislý řez“ tabulkou

Relační algebra

- projekce $R[C]$ relace se schematem $R(\mathbf{A})$ na množinu atributů C , kde $C \subseteq \mathbf{A}$

- $R(X, Y, Z)$

X	Y	Z
1	2	3
1	2	4
2	5	9

- $R[C]$, $C = \{X, Y\}$

X	Y
1	2
2	5

? co je klíčem těchto relací ?

Relační algebra

- selekce $R(\varphi)$ relace se schematem $R(\mathbf{A})$ podle logické podmínky φ
- je-li R relace, pak $R(\varphi) = \{ u \mid u \in R \wedge \varphi(u) \}$
- vytvoří relaci s tímž schematem a n-ticemi, které splňují podmínku φ
- podmínka φ je zadána boolským výrazem, obvyklé spojky and, or, not
- přijmeme-li na chvíli analogii relace a tabulky, jde o „vodorovný řez“ tabulkou

Relační algebra

- selekce $R(\varphi)$ relace se schematem $R(\mathbf{A})$ podle logické podmínky φ
- $R(X, Y, Z)$

X	Y	Z
1	2	3
1	2	4
2	5	9

- $R(\varphi)$, $\varphi: X < 2$

X	Y	Z
1	2	3
1	2	4

Relační algebra

- přiroz. spojení relací R, S se schématy $R(\mathbf{A}), S(\mathbf{B})$
 - jsou-li R a S relace, pak spojení relací
$$R * S = \{ u \mid u[\mathbf{A}] \in R \wedge u[\mathbf{B}] \in S \}$$
 - operace vytvoří největší relaci se schématem $\mathbf{A} \cup \mathbf{B}$ a n -ticemi, jejichž projekce na $\mathbf{A}$ je z relace R a projekce na $\mathbf{B}$ je z relace S
 - společné atributy $(\mathbf{A} \cap \mathbf{B})$ jsou zastoupeny pouze jednou – **vazební sloupce**
 - do výsledku jdou pouze ty n -tice z R , k nimž se v S najde n -tice se stejnými hodnotami společných atributů
 - je-li $(\mathbf{A} \cap \mathbf{B})$ prázdná množina, vytvoří se kartézský součin, kdy se každá n -tice z R spojí z každou n -ticí z S , ve výsledku je tedy $|R| * |S|$ prvků, kde $|M|$ značí mohutnost množiny M

Relační algebra

- přirozené spojení R a S

■ $R(X, Y, Z), S(A, B, Z)$

X	Y	Z
1	2	3
1	2	4
2	5	9

vazební sloupce

A	B	Z
4	strom	3
2	keř	8
24	bylina	3
16	houba	9

- $R * S$ – spojíme ty řádky z tabulek, které mají stejný obsah vazebního sloupce

X	Y	Z	A	B
1	2	3	4	strom
1	2	3	24	bylina
2	5	9	16	houba

Relační algebra

- přirozené spojení R a S, příklad 2
- $R(X, Y), S(A, B)$

X	Y
1	2
3	4

A	B
hrušeň	strom
tavolník	keř

- $R * S = R \times S$

X	Y	A	B
1	2	hrušeň	strom
1	2	tavolník	keř
3	4	hrušeň	strom
3	4	tavolník	keř

Relační algebra

■ příklad na relace a dotaz

■ relace

- KINO(název_k, adresa)
- FILM(jméno_f, herec, rok)
- PROGRAM(název_k, jméno_f, datum)

■ integritní omezení

- cizí klíče (název_k, jméno_f v relaci PROGRAM)
- jeden film nemohou v jednom kině dávat vícekrát
 - je-li např. dvojice (Blaník, Top Gun) klíčem některé n-tice v relaci PROGRAM, může se vyskytnout pouze jednou, tedy i datum může být jen jedno
- další možná IO, která neplynou ze struktury relací
 - v kinech se nehraje více než dvakrát týdně
 - jeden film se nedává více než ve třech kinech v městě

Relační algebra

- příklad na relace a dotaz

- dotaz

- najdi herce, kteří hrají ve filmech, dávaných v kině Mír

KINO	NÁZEV_K	ADRESA
	Blaník	Václ.n. 4
	Vesna	V olšinách 6
	Mír	Starostrašnická 3
	Domovina	V dvorcích
	Aero	Biskupcova

FILM	JMÉNO_F	HEREC	ROK
	Černí baroni	Vetchý	94
	Černí baroni	Landovský	94
	Top gun	Cruise	86
	Top gun	McGillis	86
	Kmotr	Brando	72
	Nováček	Brando	90
	Vzorec	Brando	80

PROGRAM	NÁZEV_K	JMÉNO_F	DATUM
	Blaník	Top gun	29.03.94
	Blaník	Kmotr	08.03.94
	Mír	Nováček	10.03.94
	Mír	Top gun	09.03.94
	Mír	Kmotr	08.03.94
	Aero	Kmotr	15.03.94
	Aero	Nováček	20.03.94
	Aero	Vzorec	21.03.94

Relační algebra

- příklad na relace a dotaz
- první krok
 - najdeme si položky programu, týkající se kina Mír
 - selekce

$R1 = \text{PROGRAM}(\text{název_k} = \text{'Mír'})$

PROGRAM	NÁZEV_K	JMÉNO_F	DATUM
	Blaník	Top gun	29.03.94
	Blaník	Kmotr	08.03.94
	Mír	Nováček	10.03.94
	Mír	Top gun	09.03.94
	Mír	Kmotr	08.03.94
	Aero	Kmotr	15.03.94
	Aero	Nováček	20.03.94
	Aero	Vzorec	21.03.94

JMÉNO_F	DATUM
Nováček	10.03.94
Top gun	09.03.94
Kmotr	08.03.94

Relační algebra

■ příklad na relace a dotaz

■ druhý krok

- výsledek R1 obsahuje názvy filmů, ale nikoli herce
- ke každé položce programu kina Mír tedy přiřadíme odpovídající detaily filmu – spojení

$$R2 = R1 * FILM$$

R1	NÁZEV_K	JMÉNO_F	DATUM
	Mír	Nováček	10.03.94
	Mír	Top gun	09.03.94
	Mír	Kmotr	08.03.94

FILM	JMÉNO_F	HEREC	ROK
	Černí baroni	Vetchý	94
	Černí baroni	Landovský	94
	Top gun	Cruise	86
	Top gun	McGillis	86
	Kmotr	Brando	72
	Nováček	Brando	90
	Vzorec	Brando	80

R2	NÁZEV_K	JMÉNO_F	DATUM	HEREC	ROK
	Mír	Top gun	09.03.94	Cruise	86
	Mír	Kmotr	08.03.94	Brando	72
	Mír	Nováček	10.03.94	Brando	90
	Mír	Top gun	09.03.9	McGillis	86

Relační algebra

- příklad na relace a dotaz

- třetí krok

- výsledek R2 obsahuje i sloupce, které nepotřebujeme
- omezíme se pouze na atribut *herec* – projekce

$R3 = R2[\text{herec}]$

- Brando byl v relaci R2 dvakrát, v relaci R3 je již jen jednou
 - množinový přístup – eliminování duplicitních prvků

R2	NÁZEV_K	JMÉNO_F	DATUM	HEREC	ROK
	Mír	Top gun	09.03.94	Cruise	86
	Mír	Kmotr	08.03.94	Brando	72
	Mír	Nováček	10.03.94	Brando	90
	Mír	Top gun	09.03.9	McGillis	86

R3	HEREC
	Cruise
	Brando
	McGillis

Relační algebra

- příklad na relace a dotaz
- řešení bez mezivýsledků

(PROGRAM(název_k='Mír') * FILM) [herec]

Relační algebra

- další užitečné relační operace

- **Θ -spojení**, kde $\Theta \in \{<, >, =, \leq, \geq, \neq\}$, $R(\mathbf{A})$, $S(\mathbf{B})$

- $R [t_1 \Theta t_2] S = \{ u \mid u[\mathbf{A}] \in R \wedge u[\mathbf{B}] \in S \wedge u.t_1 \Theta u.t_2 \}$
 - výsledná relace bude mít schema obsahující atributy z R i S (vč. duplicitních atributů), na jejich prvcích bude splněna podmínka daná v závorkách $\square$
 - kolize jmen atributů řešíme např. přejmenováním

Relační algebra

■ Θ -spojení R a S, příklad

■ $R(X, Y, Z), \quad S(A, B, Z)$

X	Y	Z
1	2	3
1	2	4
2	5	9

A	B	Z
4	strom	3
2	keř	8
24	bylina	3
16	houba	9

■ $R [Y \geq A] S$

X	Y	R.Z	A	B	S.Z
2	5	9	4	strom	3
1	2	3	2	keř	8
1	2	4	2	keř	8
2	5	9	2	keř	8

Relační algebra

- další užitečné relační operace

- **Θ -spojení**

- je ekvivalentní kartézskému součinu R a S s následnou selekcí podle podmínky
 - často používané je **spojení přes rovnost**, jehož speciálním případem je přirozené spojení (tam se vyžaduje rovnost všech odpovídajících si atributů)
 - umožňuje spojovat přes atributy se stejnou doménou, ale různým jménem
 - umožňuje spojovat přes jinou množinu atributů než v přirozeném spojení

Relační algebra

- další užitečné relační operace
 - **levé Θ -polospojení** (semijoin), $R(\mathbf{A})$, $S(\mathbf{B})$
 - $R \lt_{t_1} \Theta t_2] S = (R [t_1 \Theta t_2] S)[\mathbf{A}]$
 - definice nemá naznačit způsob implementace
 - použití např. v distribuovaných DB, aby se nemusely přenášet celé relace
 - je to vlastně omezení relace R na ty prvky, které jsou spojitelné s prvky relace S podle podmínky
 - nejčastější je polospojení přes rovnost
 - analogicky zavedeme pravé polospojení
 - lze definovat také **přírozené polospojení**, kde podmínka je rovnost všech odpovídajících si atributů
 - značíme $R \lt^* S$ resp. $R \gt^* S$

Relační algebra

- levé Θ -spojení R a S, příklad

- $R(X, Y, Z), \quad S(A, B, Z)$

X	Y	Z
1	2	3
1	2	4
2	5	9

A	B	Z
4	strom	3
2	keř	8
24	bylina	3
16	houba	9

- $R \ltimes_{Y=A} S$

X	Y	Z
1	2	3
1	2	4

Příklady na dotazy

- opět použijeme relace
 - KINO(název_k, adresa)
 - FILM(jméno_f, herec, rok)
 - PROGRAM(název_k, jméno_f, datum)
- vypiš názvy kin, která něco hrají
 - metoda: stačí prohlížet relaci PROGRAM, neboť tam všechna hrající kina jsou
 - PROGRAM[název_k]
- vypiš názvy kin, která hrají Top Gun
 - metoda: v relaci PROGRAM najít n-tice s jménem filmu Top Gun a omezit se pouze na názvy kin
 - PROGRAM(jméno_f='Top Gun')[název_k]

Příklady na dotazy

- vypiš jména filmů, které hraje kino s adresou Zvonková
 - metoda:
 - nestačí již prohlížet pouze relaci PROGRAM, neboť tam nemáme adresy kin
 - ke každé n-tici z relace PROGRAM musíme přiřadit odpovídající detaily o kinu z relace KINO – spojení
 - nebo „naopak“ – ke kinu s adresou Zvonková přiřadit jeho položky programu
 - spojení je komutativní, takže to je lhostejné
 - pozor na pořadí operací (závorky) !

`(KINO(adresa='Zvonková') * PROGRAM)[jméno_f]`

Příklady na dotazy

- vypiš názvy kin, která hrají něco s hercem Brando
 - metoda:
 - nestačí prohlížet pouze relaci PROGRAM, neboť tam nemáme jména herců
 - ke každé n-tici z relace PROGRAM musíme přiřadit odpovídající detaily o filmu z relace FILM – spojení
 - nebo „naopak“ – k filmům s hercem Brando přiřadit jeho položky programu

$(\text{FILM}(\text{herec}=\text{'Brando'}) * \text{PROGRAM}) [\text{název_k}]$

Příklady na dotazy

- dotazy, které vedou na kvantifikátory $\exists$ a $\forall$
 - dotazy, obsahující slova „všechny“, „žádný“, „každý“, apod.
 - existenční kvantifikátor $\exists$
 - v přímé (neznegované) formě odpovídá vlastně běžnému dotazu
 - univerzální kvantifikátor $\forall$
 - nutno převést na existenční kvantifikátor (viz TZI)
 - $\forall x (P(x))$ je ekvivalentní $\neg \exists x (\neg P(x))$
 - zbývá tedy vyřešit, co s „ $\neg \exists$ “
 - často je nutné vytvořit universum (všechny možné prvky) a z něj odstranit prvky, které *mají* danou vlastnost – zbydou prvky, které vlastnost nemají

Příklady na dotazy

- vypiš názvy kin, která nic nehrají
 - metoda:
 - tuto „negativní“ informaci v databázi přímo nemáme
 - je třeba použít rozdíl množin
 - od seznamu všech kin (univerzum) odečteme kina, která *něco hrají*
 - zbydou ta, která nehrají nic

KINO[název_k] – PROGRAM[název_k]

- pozor na kompatibilitu operandů (stejné atributy) !
- špatně:

KINO – PROGRAM[název_k]

Příklady na dotazy

- vypiš názvy kin, která nic nehrají
 - jiná metoda:
 - vybereme kina, jejichž název není obsažen v relaci PROGRAM

KINO(název_k $\notin$ PROGRAM[název_k]) [název_k]

- ono je to ale totéž, protože rozdíl množin $A - B$ je definován jako

$$A - B = \{x \mid x \in A \wedge x \notin B\}$$

- v mnoha klonech SQL lze zapsat tento dotaz oběma způsoby (a často i dalšími způsoby...)

Příklady na dotazy

- vypiš názvy kin, která nehrají žádný film s hercem Brando
 - např. metoda rozdíl množin
 - od seznamu všech kin (univerzum) odečteme kina, které *hrají něco s Brandem*

KINO[název_k] – (FILM(herec='Brando') * PROGRAM) [název_k]

- výsledkem budou i kina, která nehrají vůbec nic
- názvy kin, která **hrají**, ale nehrají žádný film s Brando:

PROGRAM[název_k] – (FILM(herec='Brando') * PROGRAM)
[název_k]

Příklady na dotazy

- vypiš názvy kin, která nehrají *některý* film s hercem Brando (tj. nehrají aspoň jeden z nich)

- metoda rozdílu množin

- univerzum: seznam všech kombinací {kino, „film s Brandem“}

$U = \text{KINO}[\text{název_k}] \times \text{FILM}(\text{herec}=\text{'Brando'})[\text{jméno_f}]$

- od univerza odečteme skutečné (tj. v databázi existující) kombinace {kino, jakýkoli film}

$U - \text{PROGRAM}[\text{název_k}, \text{jméno_f}]$

- není nutno odečítat jen filmy s Brandem, můžeme odečítat všechny filmy – nakreslete si třeba Venův diagram

- omezíme se pouze na název kina, tj. celý dotaz bude

$R = ((\text{KINO}[\text{název_k}] \times \text{FILM}(\text{herec}=\text{'Brando'})[\text{jméno_f}]) - \text{PROGRAM}[\text{název_k}, \text{jméno_f}])$

$[\text{název_k}]$

Příklady na dotazy

- vypiš názvy kin, která hrají všechny filmy s hercem Brando
 - kina, která něco hrají a zároveň není pravda, že nehrají některý film s Brandem
 - vezmeme kina s programem a od nich odečteme dotaz z předchozího snímku (R)

PROGRAM[název_k] – R

- jelikož dotaz R obsahuje i kina, která nehrají nic (tedy ani žádný film s Brandem), lze psát ekvivalentně též

KINO[název_k] – R

- tedy kina, u nichž neplatí, že nehrají něco s Brandem

Příklady na dotazy

- vypiš názvy kin, která hrají všechny filmy, co jsou na programu
 - kina, která něco hrají a zároveň není pravda, že nehrají některý film z programu
 - vezmeme kina s programem a od nich odečteme kina, která nehrají některý film z programu

PROGRAM[název_k] –
(((KINO[název_k] × PROGRAM[jméno_f]) –
PROGRAM[název_k, jméno_f]) [název_k])

Rozšíření relační algebry

- operace relační algebry lze rozšířit o další operace, které do rel. algebry nepatří, ale implementačně dávají smysl
 - **vnější spojení (outer join)**
 - uvažuje prázdné hodnoty buněk (proto není v RMD)
 - prázdná hodnota NULL patří do všech domén (!) a značí, že hodnota atributu je nedefinovaná, neznámá
 - NULL není nula (0), prázdný řetězec (""), apod.!
 - vede k tříhodnotové logice a k různým obtížím
 - do výsledku dává i řádky nespojitelné s ničím

Rozšíření relační algebry

■ levé vnější spojení, $R(A), S(B)$

- projekce na atributy **A** je relace R , projekce na atributy **B** je podmnožina relace S a řádek prázdných hodnot

$$R \star_L S = (R \star S) \cup (R' \times (\text{NULL}, \dots, \text{NULL}))$$

kde R' značí n -tice z R nespojitelné s S

■ pravé vnější spojení, $R(A), S(B)$

- projekce na atributy **A** je podmnožina relace R a řádek prázdných hodnot, projekce na atributy **B** je relace S

$$R \star_R S = (R \star S) \cup ((\text{NULL}, \dots, \text{NULL}) \times S')$$

kde S' značí n -tice z S nespojitelné s R

■ vnější spojení, $R(A), S(B)$

- projekce na atributy **A** je podmnožina R a řádek prázdných hodnot, projekce na atributy **B** je podmnožina S a řádek prázdných hodnot

$$R \star_F S = (R \star_L S) \cup (R \star_R S)$$

Rozšíření relační algebry

■ levé vnější spojení R a S

■ $R(X, Y, Z), S(A, B, Z)$

vazební sloupce

X	Y	Z
1	2	3
1	2	4
2	5	9

A	B	Z
4	strom	3
2	keř	8
24	bylina	3
16	houba	9

■ $R *_L S$

	X	Y	Z	A	B
♦	1	2	3	4	strom
♦	1	2	3	24	bylina
♥	2	5	9	16	houba
♣	1	2	4	NULL	NULL

Rozšíření relační algebry

- program kin včetně těch kin, která nic nehrají

KINO $\ast_L$ PROGRAM

Závěrem

- při zápisu výrazů relační algebry je vždy nutné dávat pozor na pořadí operací
- příklad:
- najdi herce, kteří hrají ve filmech, dávaných v kině Dukla
- možné řešení

$(\text{MÁ_NA_PROGRAMU}(\text{název_k}=\text{'Dukla'}) * \text{FILM}) [\text{herec}]$

- chybné řešení

$\text{MÁ_NA_PROGRAMU}(\text{název_k}=\text{'Dukla'}) * \text{FILM} [\text{herec}]$